



Enrollment System

Technical Documentation & Operations Guide

React 18

Vite

TypeScript

Tailwind CSS

Firebase Firestore

Firebase Auth

Recharts

SheetJS

Configuration • Running • Workflow • Technical Details

Version 1.0 | Prepared for project handover

Contents

- 1. System Overview
- 2. Technology Stack
- 3. Architecture
- 4. Project Structure
- 5. Configuration
- 6. How to Run
- 7. Application Workflow
- 8. Technical Details
- 9. Data Model
- 10. Security Rules
- 11. Build & Deployment
- 12. Testing & Verification
- 13. Troubleshooting
- 14. Requirement Traceability
- 15. Appendix — Quick Reference

1. System Overview

The Enrollment System is a web application that does two things:

- 1. **Collects enrollment information** from end users through a public, mobile-friendly form.
- 2. **Consolidates all submissions** into an admin dashboard with summary statistics, charts, a searchable/filterable table, and CSV/Excel export.

It is a **serverless single-page application**: the browser talks directly to Firebase Cloud Firestore, and all access control is enforced by Firestore Security Rules. There is no backend server to deploy or maintain.

Two user roles

Role	Access	Capabilities
Public user	No login required	Open the enrollment form, submit details, receive a unique reference number (e.g. <code>ENR-2026-00042</code>). Cannot read anyone's data.
Admin	Email/password login	View dashboard stats & charts, browse/search/filter/sort enrollments, open record detail, change status, export reports.

Key design principle — config-driven form

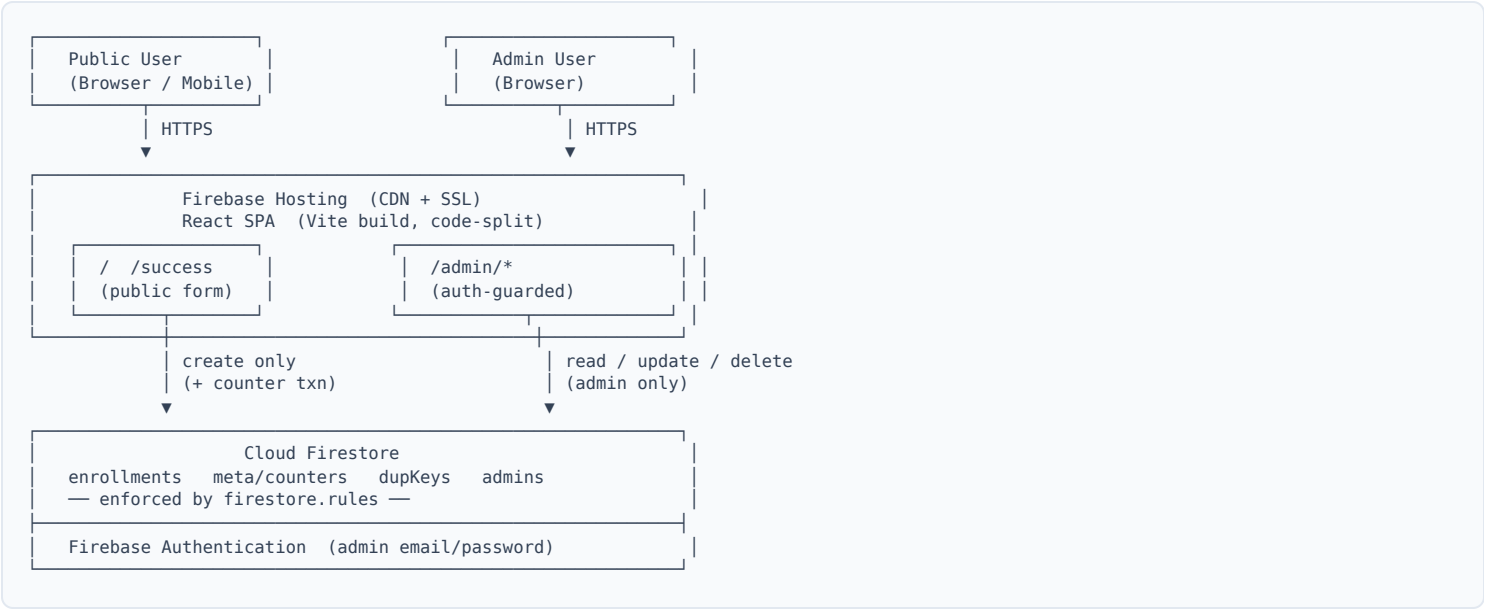
Every form field (name, label, type, validation, options) is declared in a single file, `src/config/formSchema.ts`. The public form, the validation schema, the admin table columns, and the export are all derived from it. When the field list changes, you edit that one file — nothing else.

2. Technology Stack

Layer	Technology	Why
Frontend framework	React 18 + Vite	Fast HMR dev server, modern build, lazy code-splitting
Language	TypeScript 5	Type safety across the data layer and components
Styling	Tailwind CSS 3	Rapid, consistent, responsive styling
Forms & validation	React Hook Form + Zod	Declarative, schema-derived validation
Routing	React Router DOM 6	Public + guarded admin routes
Database	Firebase Cloud Firestore	Serverless NoSQL, real-time, free tier
Authentication	Firebase Auth (email/password)	Secure admin login, no custom auth code
Charts	Recharts	Dashboard line / bar / pie charts
Export	SheetJS (xlsx)	CSV & Excel export of filtered data
Dates	date-fns	Timezone-aware formatting
Hosting	Firebase Hosting	Free SSL + CDN + custom domain
Local backend	Firebase Emulator Suite	Run Auth + Firestore locally, no cloud project needed

Requirements: Node.js 18+ (tested on 22), npm 10+, and **Java** on the PATH (the Firestore emulator is a Java process). The Firebase CLI (`firebase-tools`) is needed only for the emulator and deployment.

3. Architecture



How requests are secured

- **No server-side trust boundary is bypassed** — even though the browser writes directly to Firestore, the Security Rules reject anything that doesn't match the allowed shape and role.
- The **public bundle** only ships the enrollment form; the admin code (charts, export, table) is lazy-loaded on demand, keeping the public page fast.

4. Project Structure

```
enrollment-app/
├── src/
│   ├── config/
│   │   ├── formSchema.ts      ← SINGLE SOURCE OF TRUTH for form fields
│   │   └── validation.ts      ← Zod schema built dynamically from formSchema
│   ├── lib/
│   │   ├── firebase.ts        ← Firebase init (+ emulator wiring)
│   │   ├── enrollments.ts      ← All Firestore reads/writes, counter txn, stats
│   │   ├── export.ts           ← CSV / Excel export (SheetJS)
│   │   └── format.ts           ← Date / label / phone helpers
│   ├── context/AuthContext.tsx ← Auth state + isAdmin resolution
│   ├── hooks/useEnrollments.ts ← Paginated table data hook
│   ├── components/             ← FormField, StatCard, DataTable, Charts,
│   │                               ProtectedRoute, Spinner
│   ├── pages/
│   │   ├── EnrollPage.tsx      ← public enrollment form
│   │   ├── SuccessPage.tsx     ← reference-number screen
│   │   └── admin/
│   │       ├── LoginPage.tsx
│   │       ├── AdminLayout.tsx
│   │       ├── DashboardPage.tsx ← cards + charts
│   │       ├── EnrollmentsPage.tsx ← table, search, filter, export
│   │       └── EnrollmentDetail.tsx ← detail + status update
│   ├── types/enrollment.ts
│   ├── App.tsx                 ← routes + auth guard (admin lazy-loaded)
│   └── main.tsx
├── scripts/
│   ├── seed.mjs                ← demo data seeder (emulator)
│   ├── verify.mjs              ← security-rule + data-path tests
│   └── e2e.mjs                  ← headless-browser UI test
├── firestore.rules              ← security rules
├── firestore.indexes.json       ← composite indexes
├── firebase.json                ← Hosting + emulator config
├── .env.example                 ← environment variable template
├── README.md / ADMIN_GUIDE.md
└── package.json
```

Total application source: ~2,100 lines of TypeScript/TSX across 30 files.

5. Configuration

5.1 Environment variables

Copy `.env.example` to `.env` and fill in values. All variables are prefixed `VITE_` so Vite exposes them to the browser bundle.

Variable	Purpose	Example
<code>VITE_FIREBASE_API_KEY</code>	Firebase web API key	Alza...
<code>VITE_FIREBASE_AUTH_DOMAIN</code>	Auth domain	my-proj.firebaseio.com
<code>VITE_FIREBASE_PROJECT_ID</code>	Project ID	my-proj
<code>VITE_FIREBASE_STORAGE_BUCKET</code>	Storage bucket	my-proj.appspot.com
<code>VITE_FIREBASE_MESSAGING_SENDER_ID</code>	Sender ID	123456789
<code>VITE_FIREBASE_APP_ID</code>	App ID	1:123...:web:abc
<code>VITE_USE_EMULATOR</code>	<code>true</code> → use local emulators	false
<code>VITE_BLOCK_DUPLICATES</code>	Block repeat email/phone (FR-1.4)	true
<code>VITE_EMULATOR_HOST</code>	Emulator host (emulator mode)	127.0.0.1
<code>VITE_FIRESTORE_EMULATOR_PORT</code>	Firestore emulator port	8080
<code>VITE_AUTH_EMULATOR_PORT</code>	Auth emulator port	9099

Where to find Firebase values: **Firebase Console** → **Project Settings** → **Your apps** → **Web app** → **SDK setup and configuration**.

5.2 Configuring the form fields

Edit `src/config/formSchema.ts`. Each field is one object:

```
{
  name: 'fullName',           // dot-path; "address.city" nests into an object
  label: 'Full Name',
  type: 'text',               // text | email | tel | date | select | textarea
  required: true,
  placeholder: 'Jane Doe',
  pattern: '^[+]?[0-9 ()-]{6,20}$', // optional regex (text/tel)
  patternMessage: 'Enter a valid phone number.',
  options: [...],             // for 'select'
  section: 'Personal Details', // groups fields under a heading
  showInTable: true           // show as an admin table column
}
```

The dropdown option lists (`PROGRAM_OPTIONS`, gender, source) and the `STATUS_OPTIONS` live in the same file. Adding a program is a one-line change.

Why this matters: the client's exact field list is still being finalised. Because the form is config-driven, applying their answers is a minutes-long edit to one file, not a rewrite.

5.3 Firebase / Firestore configuration files

File	Role
<code>firebase.json</code>	Hosting target (<code>dist/</code>) + emulator ports (Auth 9099, Firestore 8080, UI 4000, Hosting 5000)
<code>firestore.rules</code>	Access-control rules (see §10)
<code>firestore.indexes.json</code>	Composite indexes for program/status + createdAt queries
<code>.firebaserc</code>	Default project alias

6. How to Run

Option A — Local with Firebase Emulator (recommended for dev/demo)

No real Firebase project required. Requires the Firebase CLI and Java.

```
# one-time
npm install
npm install -g firebase-tools      # if not already installed
cp .env.example .env              # then set VITE_USE_EMULATOR=true

# run (three terminals)
npm run emulators                 # 1) Auth + Firestore + Emulator UI
npm run seed                      # 2) create admin + 40 sample records
npm run dev                      # 3) app at http://localhost:3000
```

Then open <http://localhost:3000/>. Sign in to the admin area at **/admin/login** with the seeded account:

Field	Value
Email	<code>admin@enroll.test</code>
Password	<code>admin123</code>

Port already in use? If port 8080 (or 9099) is taken on your machine, run the emulator on alternate ports and set `VITE_FIRESTORE_EMULATOR_PORT` / `VITE_AUTH_EMULATOR_PORT` in `.env` to match. The app reads the emulator host/ports from those variables.

Option B — Real Firebase project

1. Create a Firebase project; enable **Firestore**, **Authentication** → **Email/Password**, and **Hosting**.
2. Copy the web app config into `.env`; keep `VITE_USE_EMULATOR=false`.
3. Create an admin user in Authentication → Add user.
4. In Firestore, create a document in collection `admins` with **document ID = that user's UID** and fields `{ email, role: "admin", active: true }`.
5. Deploy rules & indexes, then run the app:

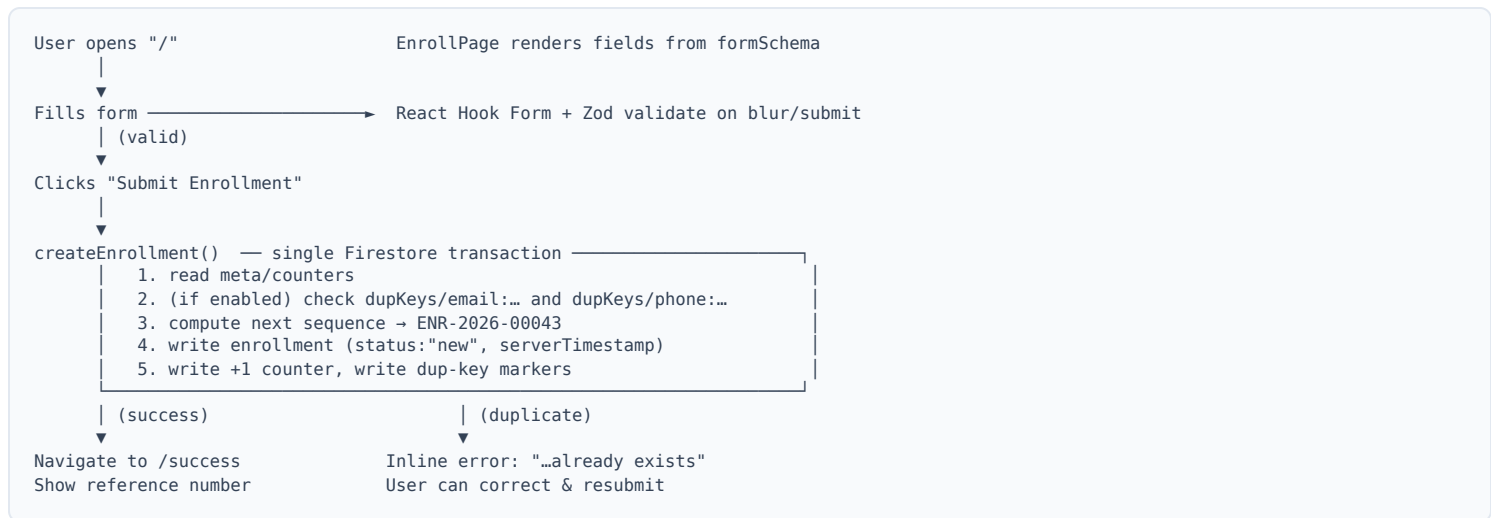
```
firebase deploy --only firestore:rules,firestore:indexes
npm run dev
```

npm scripts reference

Script	Action
<code>npm run dev</code>	Vite dev server on :3000
<code>npm run build</code>	Type-check + production build to <code>dist/</code>
<code>npm run preview</code>	Preview the production build locally
<code>npm run lint</code>	<code>tsc --noEmit</code> type-check
<code>npm run emulators</code>	Start Auth + Firestore + Hosting emulators
<code>npm run seed</code>	Seed admin user + 40 demo enrollments
<code>npm run verify:rules</code>	Run security-rule / data-path tests
<code>npm run verify:e2e</code>	Run headless-browser UI test

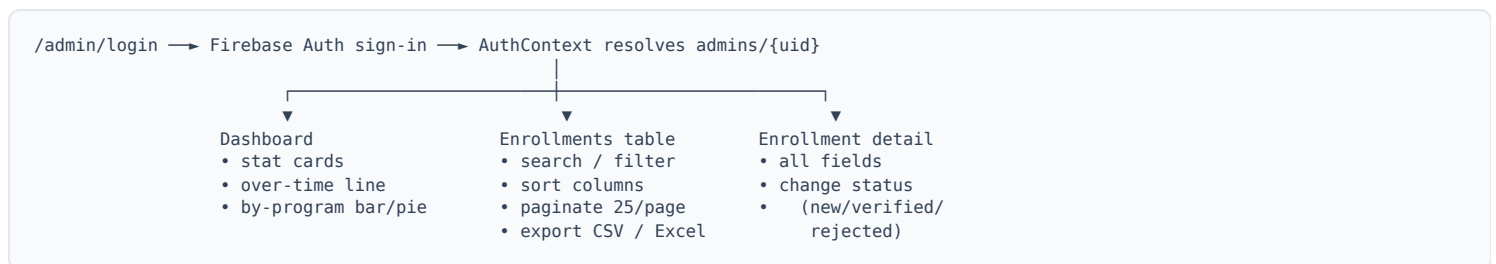
7. Application Workflow

7.1 Public enrollment flow



- **Validation (FR-1.3):** required fields, email format, phone pattern, valid date — all derived from the field config via Zod.
- **Duplicate prevention (FR-1.4):** toggled by `VITE_BLOCK_DUPLICATES`; enforced inside the transaction so it is race-safe.
- **Reference number (FR-1.5/1.6):** sequential, human-readable, shown on the success screen.

7.2 Admin flow



- **Auth guard:** `ProtectedRoute` blocks `/admin/*` unless the signed-in user has an `admins/{uid}` document.
- **Live status update:** changing a record's status writes immediately to Firestore.
- **Export respects filters:** the exported file contains exactly the records matching the current search/filter.

8. Technical Details

8.1 Config-driven form & dynamic validation

`validation.ts` walks `FORM_FIELDS` and builds a Zod object schema at runtime — including nesting dot-path names like `address.city` into sub-objects. Required string fields get a non-empty check; emails get a regex; optional fields accept empty strings. `FormField.tsx` renders the correct input (text/select/textarea/date) purely from the field's `type`.

8.2 Enrollment creation (atomic, race-safe)

`createEnrollment()` in `lib/enrollments.ts` runs everything in one `runTransaction`: all reads (counter, dup markers) happen first, then all writes. This guarantees sequential IDs and duplicate prevention even under concurrent submissions.

```
ENR-<year>-<seq:5>      // e.g. ENR-2026-00042
serverTimestamp()       // createdAt, stored in UTC
createdAt: "YYYY-MM-DD" // duplicate-free date for range queries
dupKey: "email|phone"    // composite key (audit)
```

8.3 Duplicate guard without exposing data

Public users cannot read the `enrollments` collection, so a normal "query before insert" check is impossible. Instead the app uses tiny marker documents in a `dupKeys` collection (IDs `email:...` and `phone:...`). Rules allow *existence checks* and *creation* but not listing — so duplicates are blocked without leaking who enrolled.

8.4 Pagination strategy (scales to 10k+ — NFR-5)

Situation	Strategy
No search / no date range	Cursor-based server pagination (25/page) via <code>startAfter</code> — cheap, scales indefinitely.
Search or date range active	Fetch the equality-filtered set (program/status) and paginate + full-text filter client-side — Firestore has no native text search.

8.5 Dashboard statistics

Summary cards use Firestore `getCountFromServer` aggregation (counts without downloading documents). Per-program counts are six cheap aggregation queries. The time-series chart groups the most recent records by day.

8.6 Export

`lib/export.ts` flattens each enrollment (including nested address) into a labelled row and uses SheetJS to produce `.xlsx` (with column widths) or `.csv`. The download filename is timestamped.

8.7 Auth & route guarding

`AuthContext` subscribes to `onAuthStateChanged`; on sign-in it reads `admins/{uid}` to decide `isAdmin` (and honours an `active` flag). `ProtectedRoute` redirects non-admins to the login page, preserving the intended destination.

8.8 Performance (NFR-2)

The admin bundle (Recharts + SheetJS) is lazy-loaded with `React.lazy`, so the public enrollment form downloads only what it needs. Timestamps are stored in UTC and displayed in the visitor's local timezone (NFR-6).

9. Data Model

Collection: enrollments/{autoId}

```
{
  "enrollmentId": "ENR-2026-00042",    // human-readable, sequential
  "fullName": "Jane Doe",
  "email": "jane@example.com",
  "emailLower": "jane@example.com",    // normalised for matching
  "phone": "+15551234567",
  "dob": "1995-06-15",
  "gender": "female",
  "program": "web-development",
  "source": "social-media",
  "address": { "line1": "", "city": "", "state": "", "zip": "" },
  "status": "new",                    // new | verified | rejected
  "createdAt": <serverTimestamp>,    // UTC
  "createdDate": "2026-06-11",        // for date-range queries
  "dupKey": "jane@example.com|+15551234567"
}
```

Collection: meta/counters

```
{ "enrollmentSeq": 42,
  "year": 2026 }
```

Incremented +1 inside the enrollment transaction to mint sequential IDs.

Collection: admins/{uid}

```
{ "email": "admin@client.com",
  "role": "admin",
  "active": true }
```

Document ID = Firebase Auth UID. Managed from the console.

Collection: dupKeys/{email:...|phone:...}

No-PII marker docs used to block duplicate email/phone within the enrollment transaction. Existence-checkable and creatable by the public; not listable.

Status lifecycle (FR-2.9)

new (on submission) → verified or rejected (set by an admin on the detail page).

10. Security Rules

All access control lives in `firestore.rules`. Summary of the enforced model (NFR-3):

Collection	Public (unauthenticated)	Admin
enrollments	create only , and only if the document has the required keys, a valid email, a non-empty name, <code>status == "new"</code> , and <code>createdAt == request.time</code> . Cannot read.	read / update / delete
meta/counters	read + increment, but only by exactly +1	full
dupKeys	get (existence) + create; cannot list	update / delete
admins	read own doc only; writes forbidden	read own doc (writes via console)

```
function isAdmin() {
  return request.auth != null &&
    exists(/databases/${database}/documents/admins/${request.auth.uid});
}

match /enrollments/{id} {
  allow create: if incomingIsValid();      // shape + email + status:"new"
  allow read, update, delete: if isAdmin();
}
match /meta/counters {
  allow update: if request.resource.data.enrollmentSeq
    == resource.data.enrollmentSeq + 1;    // +1 only
}
```

Verified: 8 automated rule tests confirm anonymous users cannot read enrollments, cannot write `admins`, cannot create non-`"new"` records, and cannot jump the counter by more than one — while admins can read. See §12.

11. Build & Deployment

Production build

```
npm run build      # type-checks, then builds to dist/  
npm run preview    # optional: serve dist/ locally to verify
```

The build is code-split — example output:

Chunk	Loaded for
<code>index-*.js</code> (React, Firebase, form)	every page
<code>DashboardPage-*.js</code> (Recharts)	admin dashboard only
<code>EnrollmentsPage-*.js</code> (SheetJS)	admin table only

Deploy to Firebase Hosting

```
firebase login  
firebase use <your-project-id>  
npm run build  
firebase deploy --only hosting,firestore:rules,firestore:indexes
```

Hosting serves the SPA with a catch-all rewrite to `index.html`; HTTPS is automatic. Connect a custom domain in the Firebase Hosting console.

12. Testing & Verification

Three verification suites ship with the project (run against the emulator).

Security-rule & data-path tests — `npm run verify:rules`

Uses the Firebase SDK to assert the rules behave correctly. **Result: 8/8 passing.**

- anonymous CANNOT read enrollments
- anonymous CAN create a valid enrollment
- create with missing fullName is rejected
- create with status ≠ "new" is rejected
- anonymous CANNOT write to admins
- counter cannot jump by +2 / +1 increment allowed
- admin CAN read enrollments

End-to-end UI test — `npm run verify:e2e`

Drives the real UI in headless Chrome. **Result: 7/7 passing.**

- enroll page renders
- submission succeeds with a sequential reference number
- duplicate email is blocked
- admin login → dashboard shows totals
- enrollments table renders paginated rows
- search finds the new enrollment
- no uncaught page errors

The e2e test needs a Chrome/Chromium binary; set `CHROME_PATH` if it isn't at `/usr/bin/google-chrome`.

Manual test checklist

Form validation per field · duplicate blocked · ID uniqueness under load · anonymous read denied · admin login (wrong password, logout, session) · pagination >100 records · filters/search correctness · export opens in Excel and matches the view · mobile rendering at 360px · timezone display.

13. Troubleshooting

Symptom	Cause / Fix
Emulator won't start: "port taken"	Another service holds 8080/9099. Change the ports in <code>firebase.json</code> (or a config override) and set the matching <code>VITE*_EMULATOR_PORT</code> values in <code>.env</code> .
"Could not start Firestore Emulator"	Java is not on the PATH. Install a JDK (11+).
Admin login succeeds but dashboard is blocked	The user has no <code>admins/{uid}</code> document. Create one with the user's UID (emulator mode: <code>npm run seed</code> does this).
Dashboard/table empty	No data yet. Submit via the form, or run <code>npm run seed</code> in emulator mode.
"Missing or insufficient permissions"	Rules not deployed, or reading as a non-admin. Deploy rules; confirm the admin doc exists.
Query needs an index	Deploy <code>firestore.indexes.json</code> , or click the console link in the error to auto-create.
Submission blocked unexpectedly	Duplicate guard matched an existing email/phone. Set <code>VITE_BLOCK_DUPLICATES=false</code> to allow repeats.

14. Requirement Traceability

Requirement	Implementation
FR-1.1–1.6 enrollment form & reference no.	<code>pages/EnrollPage.tsx</code> , <code>SuccessPage.tsx</code> , <code>lib/enrollments.ts</code>
FR-1.3 client validation	<code>config/validation.ts</code> (Zod from schema)
FR-1.4 duplicate prevention	transaction guard + <code>dupKeys</code> rules
FR-1.5 sequential unique ID	<code>meta/counters</code> transaction
FR-2.1 admin auth	<code>context/AuthContext.tsx</code> , <code>ProtectedRoute.tsx</code>
FR-2.2 / 2.8 dashboard & charts	<code>admin/DashboardPage.tsx</code> , <code>components/Charts.tsx</code>
FR-2.3–2.6 table, search, filter, sort, detail	<code>admin/EnrollmentsPage.tsx</code> , <code>hooks/useEnrollments.ts</code> , <code>EnrollmentDetail.tsx</code>
FR-2.7 CSV/Excel export	<code>lib/export.ts</code>
FR-2.9 status update	status select in <code>EnrollmentDetail.tsx</code>
NFR-1 responsive	Tailwind responsive layouts
NFR-2 performance	code-splitting, lazy admin bundle
NFR-3 security	<code>firestore.rules</code> (verified)
NFR-5 10k+ records	cursor pagination + count aggregation
NFR-6 UTC storage / local display	<code>lib/format.ts</code>

Optional, pending client confirmation and not built: confirmation email (Trigger Email extension, FR-1.7) and file uploads to Firebase Storage (FR-1.8).

15. Appendix — Quick Reference

Local URLs

What	URL
App	http://localhost:3000/
Admin login	http://localhost:3000/admin/login
Emulator UI	http://localhost:4000/

Demo admin credentials (emulator)

Email	admin@enroll.test
Password	admin123

One-shot local start (emulator mode)

```
npm install
npm run emulators      # terminal 1
npm run seed           # terminal 2
npm run dev            # terminal 3 → http://localhost:3000
```

Related documents

- `README.md` — setup & developer notes
- `ADMIN_GUIDE.md` — non-technical admin user guide
- `Enrollment-System-Requirement-and-Implementation.md` — original requirement